

Recurrent Neural Networks

RNNs for NLP Tasks

Introduction to Deep Learning · Class 29

Outline

Introduction

The Recurrent Language Model

Labelling and Classification

Stacking and Direction

The RNN as a Probabilistic Model

Putting It Together

Introduction

Where we are coming from

Classes 26–28

26: one cell, a state carried step to step, trained by BPTT

27: two of them, a context between, attention over the states

28: a copied cell and three gates inside the cell

what we have

a reader for sequences of any length

a reader that writes a new sequence

a reader that keeps a fact across a long wait

- Class 26 opened with four shapes of NLP problem and solved none of them; 27 solved the fourth, the encoder–decoder. The other three are today: the **next word**, a **label per token**, a **label per sequence**
- The cell is now fixed — simple or gated, it makes no difference to the wiring. What changes from task to task is **what is read out** of the states and **where the loss is paid**
- Then two ways of combining cells that every task can use: **stacking** them and running one **in each direction**

Classes 26–28; Jurafsky & Martin (2026), §14.2–14.4, §14.6.

The problems, with their examples

task	example	what is hard
next word	<i>So long and thanks for</i> → <i>all</i>	50,000 choices, from all the words before
label per token	<i>Janet will back the bill</i> → NNP MD VB DT NN	<i>back</i> : a verb here, a noun elsewhere
label per sequence	<i>a wonderfully warm little film</i> → positive	one verdict from many words, evidence anywhere
new sequence (27)	<i>the green witch arrived</i> → <i>llegó la bruja verde</i>	its own length and order

- Three of the four need a **labelled** dataset: every word tagged, every review scored. The first needs none — the next word is already in the text — which is why it can be trained on all the text there is
- Two of them decide **per token** and one **per sequence**; two need the words **after** the current one as much as the words before

Jurafsky & Martin (2026), §14.2, §14.3, §14.4.2; Class 26.

Why the reader alone is not enough

- Class 26's network reads: after t words it holds a state $\mathbf{h}_t$ that summarises them. A state is not an answer. Each task needs a **read-out** — which vectors, through which softmax — and a **loss** that says where the answer is checked
- For the next word, the read-out is at every step and the target is the text itself; for a tag, at every step against a gold tag; for a verdict, once, from the last state or from all of them together
- The reader is also **one-directional** and **one layer deep**: at *back* it has seen nothing after *back*, and it computes one transformation per word

Same cell, same training through time. The equipment of this class is what is wired around it: read-outs, losses, a second direction, a second layer.

Jurafsky & Martin (2026), §14.2–14.4; Class 26.

The approach: one recurrent layer, four output structures

- **Language model**: read the text; at every step, a softmax over the vocabulary and a loss against the next word. The text supervises itself
- **Sequence labelling**: the same, with the tag set in place of the vocabulary and gold tags as targets. One output per input, aligned
- **Sequence classification**: read the whole text; one softmax, on the last state or on a pooling of all the states, and one loss at the end. Training is **end to end**: the one error signal goes back through the classifier and the RNN
- **Stacking** feeds one RNN's outputs to the next as inputs; **bidirectionality** runs a second RNN from the end and concatenates the two states, so that the tag of *back* can see *the bill*
- The class closes with the RNN as a probabilistic model, and a tree-shaped cousin of the chain

Jurafsky & Martin (2026), §14.2–14.4, §14.6; Goodfellow et al. (2016), §10.2.3, §10.5–10.6.

Notation for today

$\mathbf{x}_t \in \{0, 1\}^{ V }$	one-hot token, index w_t ; $\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$ its embedding, $\mathbf{E} \in \mathbb{R}^{d \times V }$
d	the model dimension, shared by embeddings and states
$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$	the cell of Classes 27–28 (simple, or an LSTM with $\mathbf{c}_t$ inside)
$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$	output distribution, $\mathbf{V} \in \mathbb{R}^{ V \times d}$; $\hat{y}_t[k]$ its k -th entry
$\mathbf{h}_t^f, \mathbf{h}_t^b, [\cdot; \cdot]$	forward and backward states; concatenation
$L_{\text{CE}}, \text{PPL}$	cross-entropy loss; perplexity
$y^{(t)}, h^{(t)}, \tau$	Goodfellow's superscripts and length, in the graphical-model slides only

Notation used in this deck

Jurafsky: $\mathbf{W}$ input, $\mathbf{U}$ recurrence, $\mathbf{V}$ output, time as a subscript. Goodfellow swaps $\mathbf{U}$ and $\mathbf{W}$ and writes $y^{(t)}$. This deck keeps Jurafsky's letters, as Classes 27–28 did.

Jurafsky & Martin (2026), §14.2.1; Goodfellow et al. (2016), §10.2.

The Recurrent Language Model

The recurrent language model, recalled

Definition 1 (RNN language model)

At each step t ,

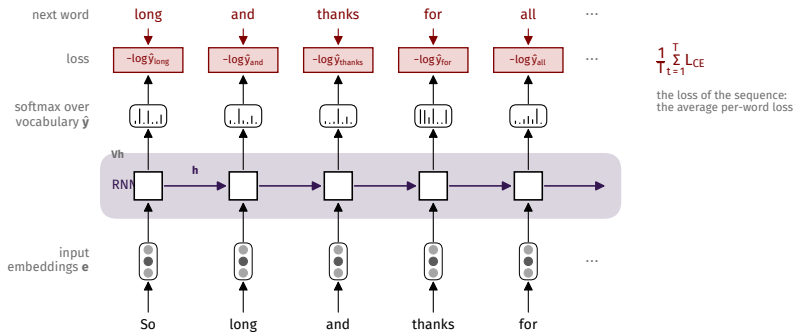
$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t),$$

and $P(w_{t+1} = k \mid w_{1:t}) = \hat{y}_t[k]$. The probability of a sequence is the product of its next-token probabilities, $P(w_{1:n}) = \prod_{i=1}^n \hat{y}_i[w_i]$.

- Against the n -gram model, the context is not $n - 1$ tokens but **everything so far**, through $\mathbf{h}_{t-1}$; against the feedforward model, not a fixed window
- Shapes: $\mathbf{E}$ is $d \times |V|$, $\mathbf{U}, \mathbf{W}$ are $d \times d$, $\mathbf{V}$ is $|V| \times d$: the vocabulary enters only at the two ends

Jurafsky & Martin (2026), Eqs. 14.4–14.9, Fig. 14.5; Mikolov et al. (2010).

Training by self-supervision



In the layout of Jurafsky's Fig. 14.6: the words go in as embeddings, the RNN carries the state, a softmax over the vocabulary at every step is compared with the word that actually came next, and each step pays $-\log \hat{y}[\text{next word}]$; the loss of the sequence is the average over its T words. No labels are added — the text is its own supervision — and the input at $t + 1$ is the true word, not the prediction (teacher forcing, Class 27). Layout of Jurafsky & Martin (2026), Fig. 14.6; §14.2.2, Eqs. 14.10–14.11.

The training loss is the log of the perplexity

Proposition 2 (Perplexity and cross-entropy)

For a text $W = w_1 \dots w_N$, the perplexity $\text{PPL}(W) = P(w_1 \dots w_N)^{-1/N}$ equals $\exp$ of the mean per-token cross-entropy:

$$\text{PPL}(W) = \exp \left(\frac{1}{N} \sum_{t=1}^N -\log P(w_t \mid w_{<t}) \right).$$

- **Proof.** By the chain rule $P(w_{1:N}) = \prod_t P(w_t \mid w_{<t})$, so $-\frac{1}{N} \log P(w_{1:N}) = \frac{1}{N} \sum_t -\log P(w_t \mid w_{<t})$; exponentiate. □
- Reading: the **effective number of choices** per token — $|V|$ for a uniform model, 1 for a perfect one; per-token, so texts of different lengths compare. In bits, $\text{PPL} = 2^{H(W)}$.

Jurafsky & Martin (2026), Eqs. 3.14–3.15, 3.42, 7.58.

The training loss is the log of the perplexity

Proposition 2 (Perplexity and cross-entropy)

For a text $W = w_1 \dots w_N$, the perplexity $\text{PPL}(W) = P(w_1 \dots w_N)^{-1/N}$ equals exp of the mean per-token cross-entropy:

$$\text{PPL}(W) = \exp \left(\frac{1}{N} \sum_{t=1}^N -\log P(w_t \mid w_{<t}) \right).$$

- Proof. By the chain rule $P(w_{1:N}) = \prod_t P(w_t \mid w_{<t})$, so $-\frac{1}{N} \log P(w_{1:N}) = \frac{1}{N} \sum_t -\log P(w_t \mid w_{<t})$; exponentiate. □
- **Reading:** the effective number of choices per token — $|V|$ for a uniform model, 1 for a perfect one; per-token, so texts of different lengths compare. In bits, $\text{PPL} = 2^{H(W)}$.

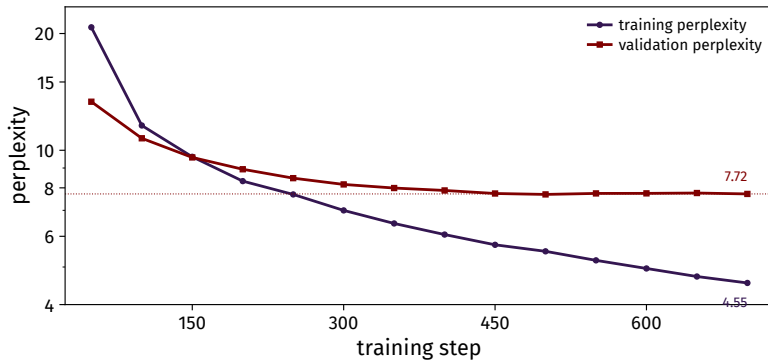
Jurafsky & Martin (2026), Eqs. 3.14–3.15, 3.42, 7.58.

A language model, trained: the experiment

- **What is being tested.** That Definition 1, trained by nothing but the text itself, learns to predict it — and that the perplexity of Proposition 2 is how to watch it
- **The data.** A public-domain story of 11,136 characters, O. Henry's *The Gift of the Magi*, the last 15% held out; at **character** level, 64 symbols, so it trains in a minute on Class 27's tape
- **The model.** An LSTM cell (Class 28) of width 64 with a tied output matrix; trained for 700 steps on windows of 48 characters, batches of 16, Adam
- **Two things to look at.** (a) The perplexity on the training and the held-out text every 50 steps: both should fall from 64 (a uniform guess) towards the effective number of choices per character. (b) Text sampled from the trained model at three temperatures, from the prefix *Della*
- **What to expect.** Training perplexity keeps falling; held-out perplexity flattens, then rises: memorising a text this small

Jurafsky & Martin (2026), §14.2.2, §14.3.3, §3.3; Class 27's tape.

A language model, trained: the perplexity



Training (indigo) and held-out (maroon) perplexity by step, log scale. Both fall fast from a uniform 64; the training curve keeps going, to 4.55, while the held-out curve flattens at 7.72 from step 450 on: from then, every further step is memorising this story rather than learning English. About 7.7 effective choices per character is what this small model learned; a word model on a large corpus reaches a perplexity in the tens over a vocabulary of tens of thousands.

Jurafsky & Martin (2026), Eqs. 3.14, 7.58.

A language model, trained: the samples

temperature 0.5

Della and could had not of the sto she was she stomed the soment and the looks, and his cruy no whis look and be greare and the stome and the shairly

temperature 1.0

Della adwade collant und haw it it thate quindouth," The contam olcoons thon wit im'll gotor or she nor nerosich shell, beat sut to it ohe a julls, be

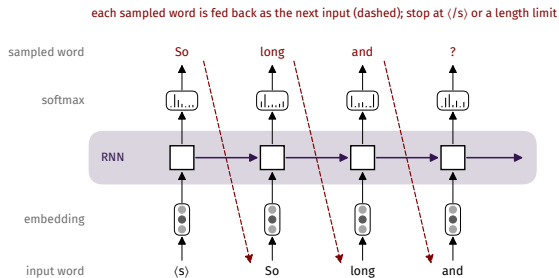
temperature 1.5

Della Rasmame1. Hef pwers, wapples pidiwat pretins stAn let," neigerony thrlitsg lages dkown it pingled lessom neslft: ancetersioth gimed leof Perion

Text sampled from the trained model, character by character, from the prefix *Della*, at three temperatures. At 0.5 the distribution is sharpened towards its likely characters: the words are mostly real, the names are the story's, and the sentences repeat. At 1.0 it is the model's own distribution: English-shaped, with invented words. At 1.5 it is flattened: spelling breaks down. None of it is in the story; the model has a distribution over characters, and sampling reads it out.

Jurafsky & Martin (2026), §14.3.3, §7.6.3.

Generation, drawn



Autoregressive generation in the layout of Jurafsky's Fig. 14.9: start from $\langle s \rangle$; the softmax gives a distribution over the next word; sample one (So); feed its embedding back as the next input; sample again (long, and, ...); stop at $\langle /s \rangle$ or a length limit. Every word is conditioned on the words the model itself chose, through the state.

Layout of Jurafsky & Martin (2026), Fig. 14.9; §14.3.3.

Generation: sampling, one token at a time

- **Autoregressive generation:** feed $\langle s \rangle$, sample a word from $\hat{y}_1$, feed its embedding back, sample again; stop at $\langle /s \rangle$ or a length limit. The text is a sample from $P(w_{1:n})$, not its mode
- **Temperature:** $\hat{y} = \text{softmax}(u/\tau)$; $\tau < 1$ sharpens towards the likely words, $\tau > 1$ flattens, $\tau \rightarrow 0$ is greedy decoding
- The same loop is Class 27's decoder, started from a context vector instead of $\langle s \rangle$: the priming makes it translation, summarisation or an answer

Class 27's greedy and beam decoding pick likely sequences; sampling picks typical ones. A language model is a distribution, and both are ways of reading it.

Jurafsky & Martin (2026), §14.3.3, Fig. 14.9, §7.6.2–7.6.3, Eq. 7.51; Shannon (1951).

Weight tying

Definition 3 (Tied language model)

The columns of $\mathbf{E}$ and the rows of $\mathbf{V}$ are both word vectors; a *tied* model uses one matrix for both:

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t).$$

Proposition 4 (What tying saves)

The embedding and output layers cost $|V| d$ instead of $2|V| d$: for $|V| = 50,000$, $d = 512$, that is 25.6M parameters fewer, with the recurrent core $2d^2$ unchanged.

- **Proof.** $\mathbf{E}$ and $\mathbf{V}$ have $|V|d$ entries each; the tied model keeps one. □
- The score of word k is $\mathbf{e}_k^\top \mathbf{h}_t$: the next word is the one whose embedding the state points at.

Jurafsky & Martin (2026), §14.2.3, Eqs. 14.12–14.14; Press & Wolf (2017); Inan et al. (2017).

Weight tying

Definition 3 (Tied language model)

*The columns of $\mathbf{E}$ and the rows of $\mathbf{V}$ are both word vectors; a **tied** model uses one matrix for both:*

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t).$$

Proposition 4 (What tying saves)

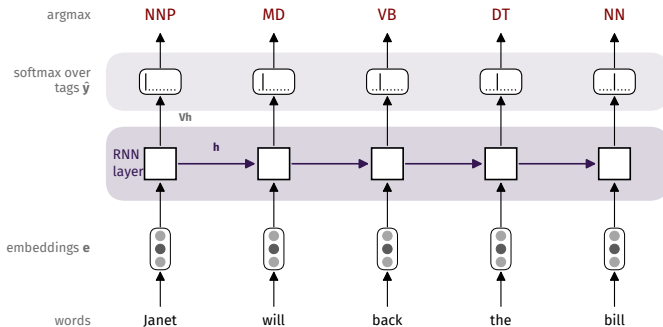
The embedding and output layers cost $|V| d$ instead of $2|V| d$: for $|V| = 50,000$, $d = 512$, that is 25.6M parameters fewer, with the recurrent core $2d^2$ unchanged.

- Proof. $\mathbf{E}$ and $\mathbf{V}$ have $|V|d$ entries each; the tied model keeps one. □
- **The score of word k is $\mathbf{e}_k^\top \mathbf{h}_t$: the next word is the one whose embedding the state points at.**

Jurafsky & Martin (2026), §14.2.3, Eqs. 14.12–14.14; Press & Wolf (2017); Inan et al. (2017).

Labelling and Classification

Sequence labelling: a label at every step



Part-of-speech tagging in the layout of Jurafsky's Fig. 14.7: the words go in as embeddings, the RNN layer carries the state, and at every step a softmax over the tag set gives a distribution whose argmax is the tag — the language model's output layer with the tag set in place of the vocabulary, a cross-entropy at every step against the gold tag, and a local decision at each. The histograms and tags are those of the left-to-right tagger trained here on a small grammar with 9 tags.

Layout of Jurafsky & Martin (2026), Fig. 14.7; §14.3.1.

Sequence classification: one label per sequence

Definition 5 (Read-outs for classification)

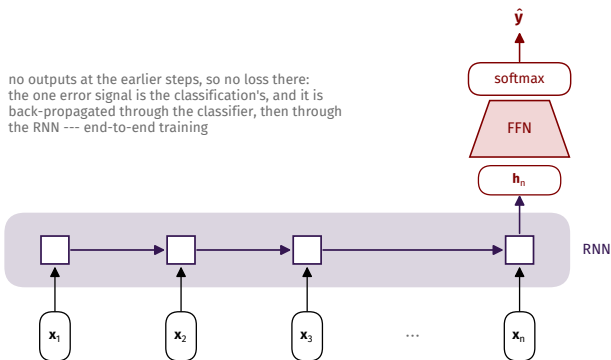
Run the RNN over $x_1 \dots x_n$ and pass one vector to a feedforward classifier with a softmax over the classes; three choices of the vector:

$$\mathbf{h}_n \text{ (the last state),} \quad \mathbf{h}_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i, \quad \mathbf{h}_{\text{max}} = \max_i \mathbf{h}_i \text{ (elementwise).}$$

- No loss at earlier positions: the one error signal goes back through the classifier, then the RNN — **end-to-end training**. $\mathbf{h}_n$ must carry the whole text; pooling reads every state

Jurafsky & Martin (2026), §14.3.2, Eq. 14.15, Fig. 14.8.

Sequence classification, drawn



In the layout of Jurafsky's Fig. 14.8. The tokens $x_1 \dots x_n$ go into one RNN; nothing is read out until the end. The final state h_n — the compressed representation of the whole text — is the input to a feedforward network (the trapezoid: a hidden layer that narrows) whose softmax gives the class $\hat{y}$. There is one loss, at the top, and its gradient goes down through the softmax and the FFN, into h_n , and back through every step of the RNN: end-to-end training. Pooling replaces h_n by the mean or the max of all the states. Layout of Jurafsky & Martin (2026), Fig. 14.8; §14.3.2, Eq. 14.15.

Why the read-out matters

Proposition 6 (Gradient paths of the three read-outs)

Let L be the classification loss. For the last-state read-out the gradient reaches $\mathbf{h}_t$ only through $n - t$ recurrent steps, $\partial L / \partial \mathbf{h}_t = (\partial L / \partial \mathbf{h}_n) \prod_{s=t+1}^n \mathbf{J}_s$; for the mean it also arrives directly, with weight $1/n$; for the max, with weight 1 on every coordinate where $\mathbf{h}_t$ attains the max.

- **Proof.** Chain rule along the unrolled graph: the last state has one path back, the recurrence; the pooled read-outs add an edge from every $\mathbf{h}_t$ to the read-out, whose derivative is $\frac{1}{n} \mathbf{I}$ or a 0/1 mask. □
- By Class 28, the product $\prod \mathbf{J}_s$ shrinks geometrically for a simple cell; the direct edge does not. If *wonderfully warm* sits at the start of a long review, the last-state classifier may never see it during training.

Jurafsky & Martin (2026), §14.3.2, in words; Class 28, Proposition 2.

Why the read-out matters

Proposition 6 (Gradient paths of the three read-outs)

Let L be the classification loss. For the last-state read-out the gradient reaches $\mathbf{h}_t$ only through $n - t$ recurrent steps, $\partial L / \partial \mathbf{h}_t = (\partial L / \partial \mathbf{h}_n) \prod_{s=t+1}^n \mathbf{J}_s$; for the mean it also arrives directly, with weight $1/n$; for the max, with weight 1 on every coordinate where $\mathbf{h}_t$ attains the max.

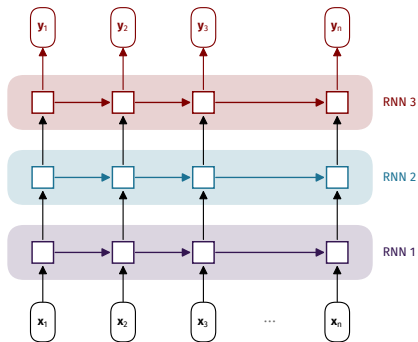
- Proof. Chain rule along the unrolled graph: the last state has one path back, the recurrence; the pooled read-outs add an edge from every $\mathbf{h}_t$ to the read-out, whose derivative is $\frac{1}{n} \mathbf{I}$ or a 0/1 mask. □
- By Class 28, the product $\prod \mathbf{J}_s$ shrinks geometrically for a simple cell; the direct edge does not. If *wonderfully warm* sits at the start of a long review, the last-state classifier may never see it during training.

Jurafsky & Martin (2026), §14.3.2, in words; Class 28, Proposition 2.

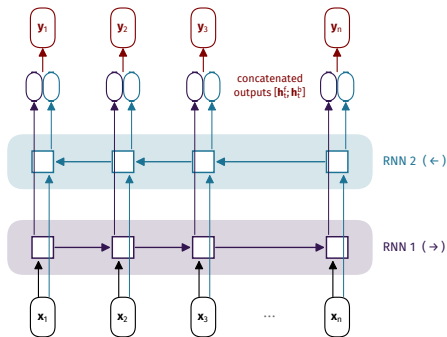
Stacking and Direction

RNNs as modules

(a) stacked: the output sequence of one layer is the input of the next



(b) bidirectional: one network each way, states concatenated



Unrolled, an RNN is a feedforward graph with vectors in and out, so it composes like any other layer; in the layouts of Jurafsky's Figs. 14.10 and 14.11.

(a) Stacked: the entire output sequence of RNN 1 is the input sequence of RNN 2, and so on; the top layer's outputs are the final outputs. (b)

Bidirectional: RNN 1 reads left to right, RNN 2 right to left, each reading every input; at each step their two states are concatenated into one vector from which the output is computed.

Layouts of Jurafsky & Martin (2026), Figs. 14.10–14.11; §14.4.

Stacking: depth per time step

Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- **Proof.** Each layer has $U, W \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. □
- Jurafsky: lower layers induce representations the upper layers use, as early visual layers find edges for later ones to find shapes; the right depth is task-specific, and the training cost rises with it. Goodfellow's three ways to be deep are the next slide.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Stacking: depth per time step

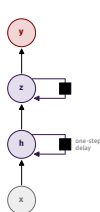
Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

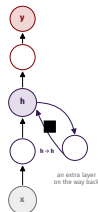
- Proof. Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. □
- Jurafsky: lower layers induce representations the upper layers use, as early visual layers find edges for later ones to find shapes; the right depth is task-specific, and the training cost rises with it. Goodfellow's three ways to be deep are the next slide.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

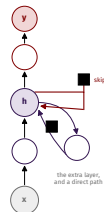
Three ways to make an RNN deep



(a) The hidden state broken into groups organised hierarchically: a lower state h with its own recurrence feeds an upper state z with its own — the stacked network of the previous slide. The lower layers turn the raw input into a representation the upper layers work with.



(b) Deeper computation in each of the three maps: an extra layer between x and h , between h and y , and — shown — on the way from h_{t-1} back to h_t . More capacity per step, but the shortest path from one time step to the next is now two layers long, which makes long dependencies harder to learn.



(c) The same, with a skip connection: a direct path from h_{t-1} to h_t beside the deeper one, so the shortest path across time is one step again. Class 28's lesson in another form: keep one path whose factors stay near one.

Layout of Goodfellow et al. (2016), Fig. 10.13, §10.5; Pascanu et al. (2014).

Bidirectionality: the right context

Definition 8 (Bidirectional RNN, as used)

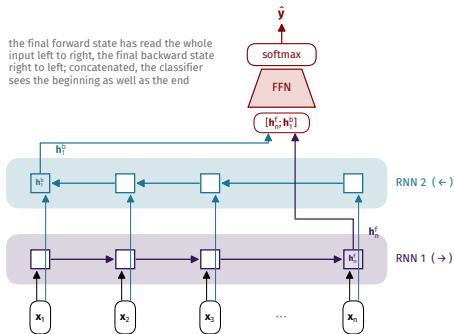
Two independent RNNs, concatenated at each step:

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(x_1, \dots, x_t), \quad \mathbf{h}_t^b = \text{RNN}_{\text{backward}}(x_t, \dots, x_n), \quad \mathbf{h}_t = [\mathbf{h}_t^f; \mathbf{h}_t^b].$$

- $\mathbf{h}_t^f$ is the ordinary state, everything to the left; $\mathbf{h}_t^b$ the state of a network run on the reversed input, everything to the right
- **Labelling**: the decision at t sees both sides. **Classification**: concatenate the two final states, so the beginning counts as much as the end. Needs the whole input, so not for generation

Jurafsky & Martin (2026), §14.4.2, Eqs. 14.16–14.18, Figs. 14.11–14.12; Schuster & Paliwal (1997); Class 26.

Bidirectional classification, drawn



In the layout of Jurafsky's Fig. 14.12. Two RNNs read the same tokens, one left to right and one right to left. The final state of each — h_n^f , which has read the whole text forwards, and h_1^b , which has read it backwards — are concatenated into one vector, and that vector is the input to the feedforward classifier of Fig. 14.8. A single final state favours the end of the text; with the backward state beside it, the beginning counts as much as the end. Layout of Jurafsky & Martin (2026), Fig. 14.12; §14.4.2.

The RNN as a Probabilistic Model

The RNN as a directed graphical model

Proposition 9 (Parameter sharing over the complete graph)

An RNN over $y^{(1)} \dots y^{(\tau)}$ with no other input defines $P(\mathbf{Y}) = \prod_{t=1}^{\tau} P(y^{(t)} | y^{(t-1)}, \dots, y^{(1)})$, with $L = \sum_t L^{(t)}$ and $L^{(t)} = -\log P(y^{(t)} | y^{(<t)})$: the complete graph over the y 's. A tabular parametrisation with k -valued y needs $O(k^{\tau})$ parameters; the RNN's count is $O(1)$ in τ .

- **Proof.** The chain rule of probability gives the product; every conditional has the whole past as parents. The table for $P(y^{(\tau)} | y^{(<\tau)})$ alone has k^{τ} entries; the RNN reuses f and θ at every step. □
- The state makes it possible: every stage has the same structure and parameters; a distant $y^{(i)}$ reaches $y^{(t)}$ through $h^{(t)}$.

Goodfellow et al. (2016), §10.2.3, Eqs. 10.29–10.33.

The RNN as a directed graphical model

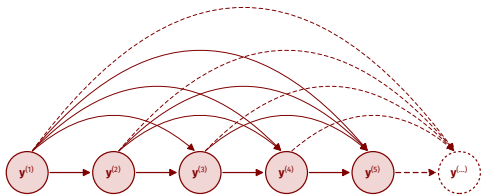
Proposition 9 (Parameter sharing over the complete graph)

An RNN over $y^{(1)} \dots y^{(\tau)}$ with no other input defines $P(\mathbf{Y}) = \prod_{t=1}^{\tau} P(y^{(t)} | y^{(t-1)}, \dots, y^{(1)})$, with $L = \sum_t L^{(t)}$ and $L^{(t)} = -\log P(y^{(t)} | y^{(<t)})$: the complete graph over the y 's. A tabular parametrisation with k -valued y needs $O(k^{\tau})$ parameters; the RNN's count is $O(1)$ in τ .

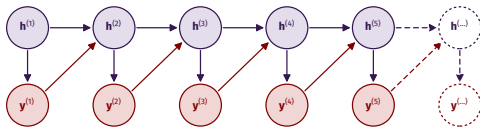
- Proof. The chain rule of probability gives the product; every conditional has the whole past as parents. The table for $P(y^{(\tau)} | y^{(<\tau)})$ alone has k^{τ} entries; the RNN reuses f and θ at every step. □
- **The state makes it possible: every stage has the same structure and parameters; a distant $y^{(i)}$ reaches $y^{(t)}$ through $h^{(t)}$.**

Goodfellow et al. (2016), §10.2.3, Eqs. 10.29–10.33.

The two graphs, drawn



(a) The complete graph over $y^{(1)}, y^{(2)}, \dots$: every past observation is a parent of every later one, so $y^{(5)}$ has four incoming edges and $y^{(t)}$ has $t - 1$. The future (dotted) is not yet observed. Parametrising this graph directly — one table per conditional — needs an ever-growing number of inputs and parameters per element.



(b) The same distribution with the state in the graph: $h^{(t)}$ has two parents, $h^{(t-1)}$ and $y^{(t-1)}$, and $y^{(t)}$ has one, $h^{(t)}$. Every stage has the same structure, so every stage can share the same parameters — $O(1)$ in the length; a distant $y^{(i)}$ still reaches $y^{(t)}$ through the states. Marginalise the states out and (a) is what remains; the price is stationarity, the same conditional at every t .

Layouts of Goodfellow et al. (2016), Figs. 10.7–10.8; §10.2.3.

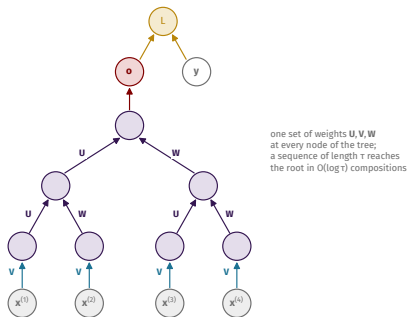
Deciding when to stop

- A model over sequences must also decide their **length**. Three mechanisms, all trained from data:
- An **end symbol** $\langle /s \rangle$ in the vocabulary, inserted after the last token of every training sequence; sampling stops when it is drawn. This is Class 27's decoder and today's language model
- A **Bernoulli output** at every step, a sigmoid trained by cross-entropy to say “continue” or “halt”; works for any output type, real-valued sequences included
- **Predict τ first**, then sample τ steps, feeding τ or $\tau - t$ back as an extra input so the model knows how close the end is:

$$P(x^{(1)}, \dots, x^{(\tau)}) = P(\tau) P(x^{(1)}, \dots, x^{(\tau)} \mid \tau)$$

Goodfellow et al. (2016), §10.2.3, Eq. 10.34; Schmidhuber (2012).

The tree-shaped cousin: recursive networks



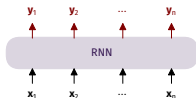
- The RNN's idea — one function, one parameter set, applied repeatedly — on a **tree**: every leaf reads an input through V , every node combines its left child through U and its right child through W , and the root gives o , compared with y in the loss L
- A variable-length sequence still maps to one vector with a fixed set of weights; leaf to root is $O(\log \tau)$ compositions for a balanced tree, against τ for the chain
- The open question is the tree: balanced, from a parser, or learned

Layout of Goodfellow et al. (2016), Fig. 10.14, §10.6; Pollack (1990); Socher et al. (2011, 2013).

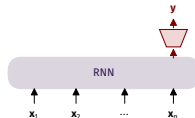
Putting It Together

Four architectures

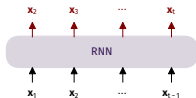
(a) sequence labelling



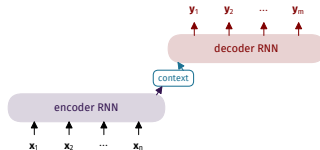
(b) sequence classification



(c) language modelling



(d) encoder--decoder

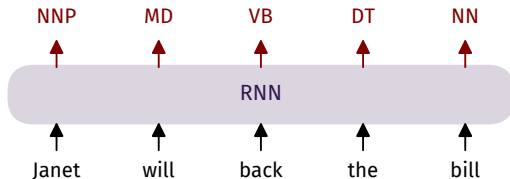


The four architectures of the block, in the layout of Jurafsky's Fig. 14.15. (a) Sequence labelling: each input token to an output token. (b) Sequence classification: the whole input to one class, through the last state or a pooled one. (c) Language modelling: the next token at every step. (d) Encoder-decoder: one RNN maps the input to a context, another maps the context to an output sequence of its own length. Each on its own, with its example, on the next four slides.

Layout of Jurafsky & Martin (2026), Fig. 14.15; §14.6.

(a) Sequence labelling

sequence labelling: one output per input, aligned

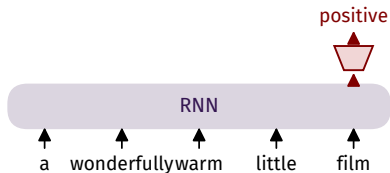


- **The problem:** assign a label to every token — a part-of-speech tag, a named-entity tag, a boundary mark. One output per input, at the same position
- **The wiring:** the RNN's state at every step goes through one softmax over the label set; the loss is the cross-entropy at every step against the gold label; the decision at t is local
- **The example:** *Janet will back the bill* → NNP MD VB DT NN. A bidirectional RNN lets the tag of *back* see *the bill*

Jurafsky & Martin (2026), §14.3.1, §14.4.2, Fig. 14.7.

(b) Sequence classification

sequence classification: one output for the whole input

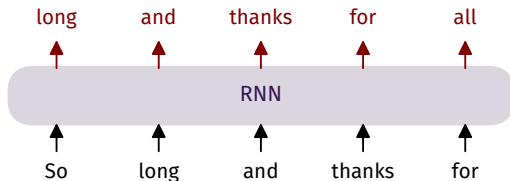


- **The problem:** one label for the whole sequence — the sentiment of a review, spam or not, the topic of a document
- **The wiring:** the RNN reads everything; the last state, or the mean or max of all the states, goes through a feedforward classifier and one softmax; one loss, at the end, trained end to end. A bidirectional RNN concatenates the two final states
- **The example:** *a wonderfully warm little film* → positive. The evidence, *wonderfully warm*, sits early; pooling or a backward state keeps it in reach

Jurafsky & Martin (2026), §14.3.2, §14.4.2, Figs. 14.8, 14.12.

(c) Language modelling

language modelling: the next token at every step

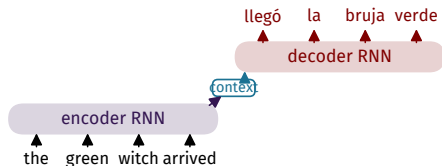


- **The problem:** the probability of the next token given all the tokens before — and, from it, of any text, and of text the model writes itself
- **The wiring:** the RNN's state at every step goes through a softmax over the whole vocabulary; the target at t is the word at $t + 1$, so no labels are needed; the loss is the average cross-entropy, and $\exp$ of it is the perplexity
- **The example:** *So long and thanks for* → *all*, with *fish* and *memories* as runners-up. Sampling from the softmax, one word at a time, generates text

Jurafsky & Martin (2026), §14.2, §14.3.3, Figs. 14.6, 14.9.

(d) Encoder-decoder

encoder--decoder: a new sequence, of its own length and order



- **The problem:** an output sequence of its own length and order — translation, summarisation, a question's answer
- **The wiring** (Class 27): an encoder RNN reads the source into a context; a decoder RNN, a language model primed with it, writes the target one token at a time; attention gives every decoder step its own context
- **The example:** *the green witch arrived* → *llegó la bruja verde* — the verb moves to the front, the adjective after the noun

Jurafsky & Martin (2026), §14.7–14.8, Figs. 14.17–14.22; Class 27.

The block, in one table

	output	loss	where the gradient enters
language model	$\hat{y}_t$ over V , every step	$-\log \hat{y}_t[w_{t+1}]$, averaged	every step
sequence labelling	$\hat{y}_t$ over tags, every step	cross-entropy, every step	every step
classification	one softmax	one cross-entropy	$\mathbf{h}_n$, or every $\mathbf{h}_t$ if pooled
encoder-decoder (27)	$\hat{y}_t$ over V , decoder steps	teacher-forced cross-entropy	decoder steps, back through $\mathbf{c}$
stacked	top layer's outputs	as above	through L cells per step
bidirectional	$[\mathbf{h}_t^f; \mathbf{h}_t^b]$	as above	from both directions

One cell, three matrices, and the choice of what to read out and where to pay the loss: that choice is the architecture.

Takeaways

1. Four problems, one reader: what differs is the read-out and where the loss is paid
2. A language model is an RNN whose output is the next token; it trains on raw text by self-supervision, and its loss is the log of its perplexity (Definition 1, Proposition 2)
3. Tying the output matrix to the embeddings removes $|V|d$ parameters (Definition 3, Proposition 4); sampling with a temperature reads the model as a distribution
4. Labelling emits a distribution at every step; classification reads out one vector, and pooling gives the early tokens a direct path to the loss (Definition 5, Proposition 6)
5. Stacking buys depth per step at the same parameter count (Proposition 7); bidirectionality buys the right context (Definition 8)
6. The RNN is the complete graphical model over the sequence with $O(1)$ parameters in its length (Proposition 9); the recursive net is the same idea on a tree

Classes 30–31: the recurrence goes, attention stays — the transformer.

References i

- Jurafsky, D. & Martin, J. H. (2026). *Speech and Language Processing*, 3rd ed. draft, August 2026. Ch. 14, §14.2–14.4, 14.6; §3.3, §3.7; §7.6.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press. §10.2.3, §10.3, §10.5–10.6.
- Mikolov, T., Karafiát, M., Burget, L., Černocký, J. & Khudanpur, S. (2010). Recurrent neural network based language model. *Interspeech*.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14, 179–211.
- Press, O. & Wolf, L. (2017). Using the output embedding to improve language models. *EACL*.
- Inan, H., Khosravi, K. & Socher, R. (2017). Tying word vectors and word classifiers. *ICLR*.
- Schuster, M. & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Trans. Signal Processing*, 45, 2673–2681.
- Graves, A., Mohamed, A. & Hinton, G. (2013). Speech recognition with deep recurrent neural networks. *ICASSP*.
- Pascanu, R., Gulcehre, C., Cho, K. & Bengio, Y. (2014). How to construct deep recurrent neural networks. *ICLR*.
- Graves, A. (2013). Generating sequences with recurrent neural networks. arXiv:1308.0850.

References ii

- Shannon, C. E. (1951). Prediction and entropy of printed English. *Bell System Technical Journal*, 30, 50–64.
- Schmidhuber, J. (2012). Self-delimiting neural networks. arXiv:1210.0118.
- Pollack, J. B. (1990). Recursive distributed representations. *Artificial Intelligence*, 46, 77–105.
- Socher, R., Lin, C. C., Ng, A. Y. & Manning, C. D. (2011). Parsing natural scenes and natural language with recursive neural networks. *ICML*.
- Socher, R. et al. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. *EMNLP*.
- Henry, O. (1905). The Gift of the Magi. Public domain; the text the language model is trained on.

Questions?